CHAPTER 1

Tree Traversal

Trees can be "traversed" in a few different ways. When we say traversing a tree, we mean that we are visiting and extracting data from each node in a tree. We classify these *traversals* based on the order in which the nodes are visited.

This chapter will discuss *breadth-first search*, also known as *BFS*, which traverses in a *level-order*. Each level is traversed in sequential order.

Alternatively, there is *depth-first search* (*DFS*), which travels as far down a branch as possible before backtracking and checking other branches. Within DFS there are 3 common traversal methodologies: *in-order*, *pre-order* and *post-order*.

While the demonstrations for these traversals will be for trees, they work the same for any given graph, under one condition: using DFS or BFS on a *graph* requires you do have a preselected node a your *start node*.

1.1 BFS

Let's look at running BFS on a tree. Starting out with this tree:

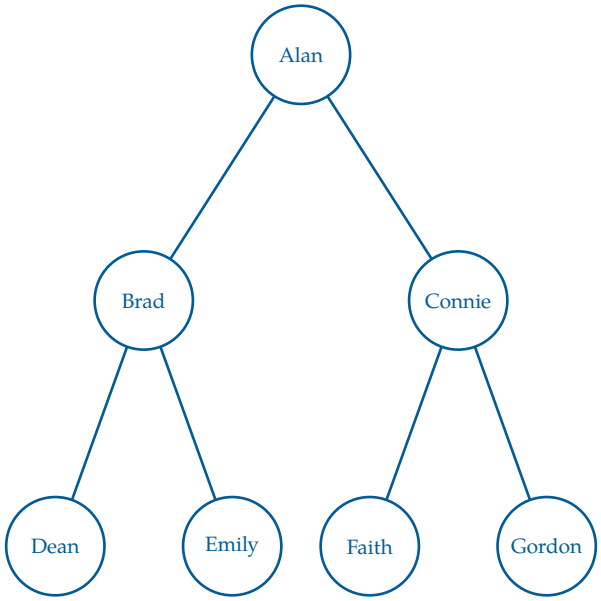


Figure 1.1: Our original tree containing a list of names to be traversed.

To perform BFS, we use a *queue*, which follows a first-in, first-out (*FIFO*) order. We begin by placing the root node into the queue. At each step, we remove the front node from the queue, visit it, and then add all of its children to the *back* of the queue.

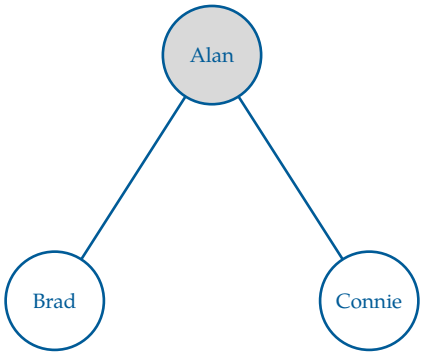


Figure 1.2: Step 1: Visit Alan, then add its children (Brad, Connie) to the queue.

Here, we visit the root node first, which is Alan. After visiting Alan, we add its children, Brad and Connie, to the queue. Since there are no other nodes to visit at this level, we move on to the next level of the tree.

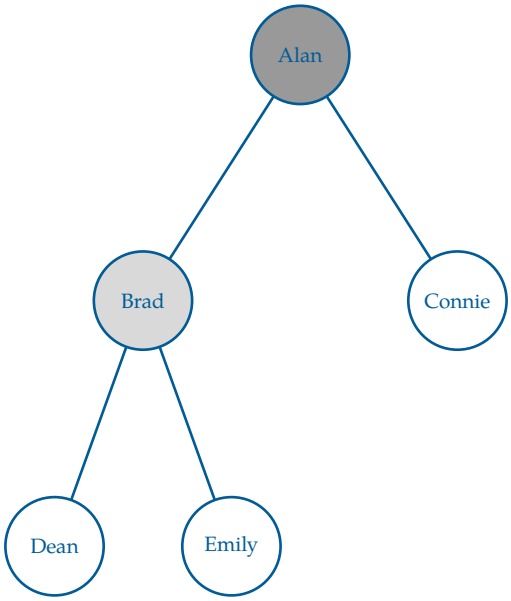


Figure 1.3: Step 2: Visit Brad, then add its children (Dean, Emily) to the queue.

We continue this process, visiting Connie next and adding its children, Faith and Gordon, to the queue. The queue now contains Connie, Dean, and Emily. Here is a step trace of the BFS traversal:

Step	Queue	Output (visited)
Start	[Alan]	[]
1	[Brad, Connie]	[Alan]
2	[Connie, Dean, Emily]	[Alan, Brad]
3	[Dean, Emily, Faith, Gordon]	[Alan, Brad, Connie]
4	[Emily, Faith, Gordon]	[Alan, Brad, Connie, Dean]

Continuing this process until the queue is empty gives the final traversal:

Alan, Brad, Connie, Dean, Emily, Faith, Gordon

This ordering reflects a level-by-level traversal of the tree, which is why BFS on a tree is also called level-order traversal.

To use BFS for *searching* rather than full traversal, we simply introduce a target value and stop the algorithm as soon as the target is found. Instead of visiting every node, we check each node when it is dequeued and terminate early if it matches the desired value.

For example, suppose we want to find "Faith" in the tree. BFS will examine nodes level by level: Alan, then Brad and Connie, then Dean, Emily, and Faith. As soon as Faith is encountered (we *dequeue* it from the queue), the search stops, without visiting any remaining nodes.

1.1.1 Pseudocode

Our steps can be reflected as pseudocode as follows:

Algorithm 1 Breadth-First Search (BFS) - Iterative

```
procedure BFS(root, target)
    Q = empty queue
    ENQUEUE(Q, root)
    while Q is not empty do
        node = DEQUEUE(Q)
        if node = target then
            return node
        end if
        for each child of node do
            ENQUEUE(Q, child)
        end for
    end while
    return null
end procedure
```

This procedure is often used in game applications to find paths, such as maze solvers, chess algorithms, and tic-tac-toe solutions. It is also used in social network analysis to find the shortest path between two users, and in web crawling to explore the structure of websites.

1.2 DFS

In contrast, we can run a *depth-first search* on the original tree (see Figure 1.1). Depth-first search, as the name implies, goes as deep down the tree until reaching a leaf, and then backtracks to the next possible, unsearched path. DFS (implicitly) uses a stack to explore the graph or tree.

The steps are fairly simple:

1. Push the root node into the stack.
2. Pop a node from the stack, and mark it as visited.
3. Push the node's children onto the stack.
4. Repeat steps 2 and 3 until the stack is empty.

There are 3 orders in which we can traverse/search the deepest tree. We will look at `pre-order`, `post-order`, and `in-order`.

However, these orders essentially perform the same basic three tasks,

V Visit the selected node.

L Run DFS on the selected node's left subtree.

R Run DFS on the selected node's right subtree.

The recursion terminates when we reach a leaf. At this point, the algorithm returns to the previous call, which is how *backtracking* occurs. This backtracking is done via the stack we had mentioned previously.

1.2.1 Pre-order

In `pre-order` traversal, we visit the node *before* exploring its subtrees. The order is:

$$(V, L, R)$$

We visit the node *before* its children because the node's value is needed *prior* to exploring

its subtrees. This is useful when the node represents a decision that affects which branch of the tree you go down, such as in tree copying or directory traversal like we talked about in the previous chapter.

Using our original tree, the pre-order results are:

Alan, Brad, Dean, Emily, Connie, Faith, Gordon

1.2.2 Post-order

In `post-order` traversal, we visit the node *after* exploring both subtrees. The order is:

$$(L, R, V)$$

We visit the node *after* its children because the node's computation depends on results from both subtrees. This is useful when combining subtree results or deleting trees, where children must be handled before the parent.

The post-order traversal of our original tree is

Dean, Emily, Brad, Faith, Gordon, Connie, Alan

1.2.3 In-order

In `in-order` traversal, we visit the node *between* exploring its subtrees. The order is:

$$(L, V, R)$$

We visit the node *between* its subtrees because the correct interpretation of the node depends on partially processed children. In-order only makes sense for binary trees, because it returns the *sorted order* of the binary search tree. It depends on left and right subchildren. This would be like the alphabetical order of our dictionary example from the previous chapter.

The in-order traversal of our original tree becomes

Dean, Brad, Emily, Alan, Faith, Connie, Gordon

1.2.4 Pseudocode

The previous sections (pre-order, in-order, and post-order) describe *when* a node is visited during a depth-first traversal.

The following algorithm instead describes the general *mechanism* of depth-first search. This implementation corresponds to a pre-order traversal. Other traversal orders (in-order and post-order) require additional state and cannot be obtained by simple reordering alone.

Algorithm 2 Depth-First Search (DFS) - Iterative on a Tree

```
procedure DFS(root)
    S = empty stack
    PUSH(S, root)
    while S is not empty do
        node = POP(S)
        visit(node)                                  ▷ V
        if node.left ≠ null then
            PUSH(S, node.left)                       ▷ L
        end if
        if node.right ≠ null then
            PUSH(S, node.right)                      ▷ R
        end if
    end while
end procedure
```

DFS is particularly useful for solving problems like connected-component detection in graphs and maze-solving.

1.3 Recursive vs Iterative Traversal

As we've established, trees are recursively defined structures, and as such, our algorithms for them can be defined as such too. This allows us to implement searches and traversals in two ways: *recursive* and *iterative*. Let's look at DFS recursively.

Importantly, they do not change the traversal order (pre-, in-, post-order); they only change how the traversal is implemented.

A *recursive traversal* relies on the program's *call stack* (recall from the Advanced Python Syntax chapter). Each time the algorithm explores a subtree, it makes a function call, and the system automatically remembers where to return when that call finishes. This matches the recursive definition of a tree, where each subtree is itself a tree. This allows us to get rid of the stack in DFS and solely rely on a computer's call stack.

Exercise 1 Recursive DFS

Take a look at the Iterative DFS code above.

How can we rewrite the pseudocode to be recursively define, taking advantage of a inherent recursive structure? Try pre-order recursively first. Assume the tree is a binary tree.

Working Space

Answer on Page 9

Note that we can create recursive DFS just by reordering the three operations (`V`, `L`, `R`).

What about BFS? BFS is typically implemented iteratively because it relies on a queue. While recursive versions exist, they still depend on a queue and offer little conceptual advantage, so we omit them here.

1.3.1 Pre-order and Post-order Numbering

Appendix A

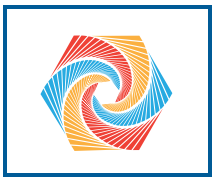
Answers to Exercises

Answer to Exercise 1 (on page 7)

Algorithm: DFS (Pre-order) run recursively on a tree

```
procedure DFS(root)
    if root = null then
        return
    end if
    visit(root)                ▷ V
    DFS(root.left)             ▷ L
    DFS(root.right)            ▷ R
end procedure
```

This algorithm defines a depth-first search on a binary tree. Because the node is visited before its subtrees, it corresponds specifically to a pre-order traversal.



Index